

Lesson 4 Intelligent Stacking

1. Program Logic

First, program ArmPi to recognize colors with Lab color space. Convert the RGB color space to Lab, image binarization, and then perform operations such as expansion and corrosion to obtain an outline containing only the target color. Use circles to frame the color outline to realize object color recognition.

Secondly, judge whether the object moves after recognition. Obtain the center coordinates of the object, transform the coordinates into reality, and then compare the last coordinates by drawing the center point to determine whether it is a movement. If the object is still moving, the robotic arm will track it. When the object stops moving for few seconds, the robotic arm will grip it.

Secondly, program to decide the order of gripping. There are 2 situations for the gripping sequence.

Case One: Place red, green and blue block in the recognition area in advance.

ArmPi will grip in the order of red, green and blue (prioritize the color that is recognized first).

Case Two: Place three same color blocks in the area in advance such as 3 red blocks. By calculating and comparing the recognition area of the objects, the object with largest area is preferred.

Last step is to recognize and stack the blocks, then back to the initial position.

The source code of the program is located in :
</home/pi/ArmPi/Functions/ColorPalletizing.py>

```
while True:
    if __isRunning:
        if detect_color != 'None' and start_pick_up: # if it is detected that the block has not removed for a period of time, start to pick up
            set_rgb(detect_color)
            setBuzzer(0.1)
            # highly cumulative
            z = z_r
            z_r += dz
            if z == 2 * dz + coordinate['red'][2]:
                z_r = coordinate['red'][2]
            if z == coordinate['red'][2]:
                move_square = True
                time.sleep(3)
                move_square = False
            result = AK.setPitchRangeMoving((world_X, world_Y, 7), -90, -90, 0) # 5cm Remove to target postion, high is 5 cm
            if result == False:
                unreachable = True
            else:
                unreachable = False
                time.sleep(result[2]/1000)

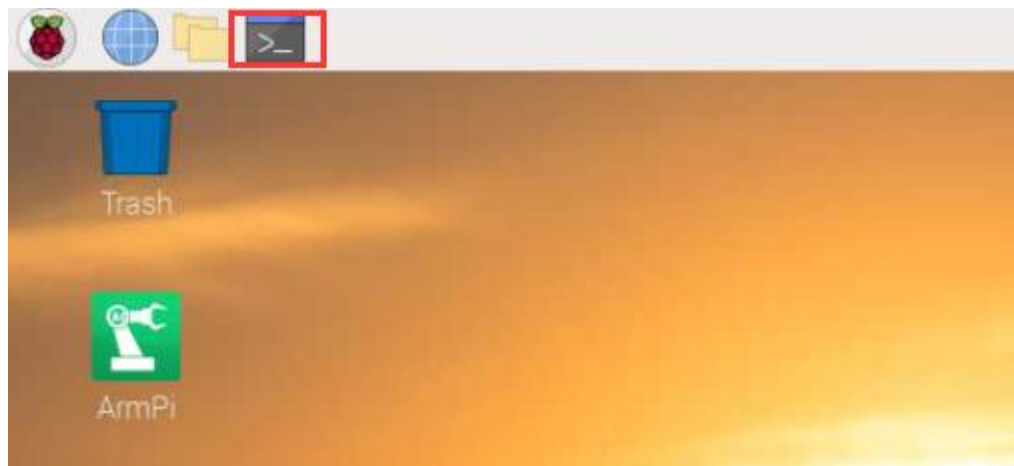
            if not __isRunning:
                continue
            # calculationthe angle at that the clamper gripper needs to rotate
            servo2_angle = getAngle(world_X, world_Y, rotation_angle)
            Board.setBusServoPulse(1, servo1 - 280, 500) # claw open
            Board.setBusServoPulse(2, servo2_angle, 500)
            time.sleep(0.5)
```

2. Operation Steps

i Pay attention to the text format in the input of instructions.

1) Turn on robotic arm and connect to Raspberry Pi desktop with VNC.

2) Click  or press “Ctrl+Alt+T” to open the LX terminal.



3) Enter “cd ArmPi/Functions/” command, and then press “Enter” to come to the category of games programmings.

```
pi@raspberrypi: ~  
File Edit Tabs Help  
pi@raspberrypi:~ $ cd ArmPi/Functions/
```

4) Enter “sudo python3 ColorPalletizing.py”, then press “Enter” to start the game.

```
pi@raspberrypi: ~/ArmPi/Functions  
File Edit Tabs Help  
pi@raspberrypi:~ $ cd ArmPi/Functions/  
pi@raspberrypi:~/ArmPi/Functions $ sudo python3 ColorPalletizing.py
```

5) If you want to exit the game programming, press “Ctrl+C” in the LX terminal interface. If the exit fails, you can press it multiple times.

3. Project Outcome

i In order to grip easily, please do not place the blocks too close.

Place three color blocks separately in the camera recognition area. When ArmPi recognizes the green block, the buzzer on the Raspberry Pi expansion board will beep, and the RGB lights will light in green. ArmPi will put the block in stacking area and then continue to stack in turn.

